

StrainSense

Full Detailed Hackathon Plan

Real-Time Movement Intelligence for Posture, Corrective Mobility, and Injury Prevention

1. What we are building

StrainSense is a real-time, camera-based corrective movement system.

It does **three core things**:

1. **Scans the user's body and movement** through their camera
2. **Detects posture, asymmetry, and instability issues**
3. **Guides them through corrective drills and stretches with live visual feedback**

This is not:

- a generic fitness app
- an LLM posture chatbot
- a static posture checker
- a med-document translator

This is:

a movement intelligence system that sees how your body is misaligned, maps that to likely strain patterns, and coaches safer correction live.

That fits the hackathon's **Biology & Physical Health** track well, since it helps users understand their physical health and access actionable movement support in a clear, bounded way. It also plays nicely with the judging emphasis on real-world impact, technical execution, ethical alignment, and presentation.

2. The actual problem

2.1 Surface-level problem

A lot of people:

- have poor posture
- move badly
- have tight hips, rounded shoulders, forward head posture, knee collapse, etc.
- do stretches wrong
- don't know what the real issue is
- don't have a PT watching them in real time

2.2 Deeper problem

There are **three layers** to the problem:

A. Detection problem

Most people cannot accurately identify what's wrong with their own movement or posture.

B. Translation problem

Even if they know "my hips feel tight" or "my shoulders are rounded," they don't know:

- what that means
- what it may lead to
- what specifically to do about it

C. Execution problem

Even after finding "helpful stretches" online, people:

- do them incorrectly
- compensate with other body parts
- reinforce bad movement patterns
- potentially increase injury risk

That means current solutions are often weak because they are:

- generic
- non-personalized
- static
- non-interactive
- not visually grounded in the user's own body

3. Product thesis

3.1 Core thesis

People do not need more generic movement advice. They need a system that watches how their body actually moves, identifies what is unstable or misaligned, and helps them correct it in real time.

3.2 Strong one-line pitch

StrainSense uses computer vision to detect postural asymmetry and unstable movement patterns, then gives live visual coaching to reduce strain and improve corrective movement in real time.

3.3 Stronger judge-facing pitch

We built a real-time injury-prevention and corrective mobility system that sees where your body is unstable, explains why it matters, and helps you fix it before strain becomes pain.

4. Final product structure

The product should have **3 major phases**.

Phase 1: Guided Movement Scan

The app asks the user to perform a short guided assessment:

- front neutral stance
- side neutral stance
- overhead arm raise
- bodyweight squat
- optional single-leg balance

The system tracks pose landmarks and detects:

- asymmetry
- posture deviations
- unstable joints
- mobility limitations
- compensation patterns

Phase 2: Body Strain Map + Corrective Plan

The app then generates:

- identified issue cards
- annotated screenshots / frames
- body-region highlights
- alignment deviations
- recommended corrective drills or stretches

Phase 3: Live Corrective Coaching

The user selects a drill.

The app:

- watches their body live
- overlays skeleton and alignment guides
- highlights target joints
- draws correction arrows
- gives cue text
- warns if form becomes unstable or unsafe

That's the full loop:

scan → **analyze** → **prescribe** → **coach**

5. Why this is hackathon-strong

5.1 It is visually impressive

Judges will literally see:

- the live body feed
- skeleton tracking

- red unstable joints
- directional arrows
- correction cues
- instant green/red feedback as the user fixes or worsens form

That's way stronger than text.

5.2 It feels advanced

It looks like:

- computer vision
- movement analytics
- live biomechanical feedback
- corrective prescription
- safety-aware coaching

5.3 It is explainable

You can clearly explain:

- pose estimation
- joint-angle calculations
- asymmetry heuristics
- issue-to-drill mapping
- live threshold logic
- safety states

That matters because the hackathon explicitly says teams must be able to explain their technical stack and architectural decisions, not just wrap AI in a shallow way.

6. What we should and should not claim

6.1 We should claim

- posture and movement assessment
- asymmetry detection
- strain-risk indicators
- corrective mobility guidance

- live form feedback

6.2 We should not claim

- diagnosis
- medical certainty
- physical therapy replacement
- injury diagnosis
- treatment efficacy guarantees

Use language like:

- “pattern consistent with”
- “tendency toward”
- “may contribute to”
- “suggested corrective focus”

That will help with ethical alignment, which the hackathon values.

7. Target user

Primary user

A student, office worker, or active person who:

- sits a lot
- has mobility issues
- suspects poor posture
- wants to improve movement quality
- wants feedback while doing corrective drills

Secondary user

Someone using:

- at-home stretching
- beginner rehab-style movement
- self-guided mobility work

This is a great audience because it's believable and easy to demo.

8. MVP scope

Do **not** support everything.

8.1 Supported assessment actions

For MVP, support:

- front stance
- side stance
- overhead arm raise
- bodyweight squat
- single-leg balance

8.2 Supported issue types

Support only 4–5 issues:

1. Rounded shoulders
2. Forward head posture
3. Anterior pelvic tilt tendency
4. Knee valgus / knee collapse in squat
5. Left-right asymmetry / weight shift

8.3 Supported drills

Support only 4–5 drills:

1. Doorway pec stretch
2. Wall angel
3. Hip flexor stretch
4. Squat alignment drill
5. Split squat / lunge control drill

That is enough to make the system feel real.

9. The full user flow

Step 1: User opens app

They see:

- what StrainSense does
- disclaimer
- camera setup guidance
- start assessment button

Step 2: Camera calibration

System checks:

- full body visible
- enough distance
- decent lighting
- body centered

Step 3: Guided movement scan

User performs:

- front stance
- side stance
- arm raise
- squat
- balance test

System tracks pose in real time.

Step 4: Analysis

System computes:

- posture metrics
- asymmetry scores
- instability flags
- movement deviations

Step 5: Results page

System shows:

- issue cards
- annotated body frames
- asymmetry / strain visuals
- recommended corrective drills

Step 6: Drill selection

User clicks a drill and sees:

- what it helps with
- key movement cues
- short demo animation/video

Step 7: Live coaching mode

User performs the drill while system shows:

- skeleton overlay
- highlighted joints
- arrows
- feedback text
- unsafe warnings
- quality score / hold timer

Step 8: Session completion

System shows:

- drill completed
 - errors corrected
 - final movement quality
 - next suggested drill
-

10. Detection logic

10.1 Input

Live webcam feed.

10.2 CV layer

Use **MediaPipe Pose** to get body landmarks.

10.3 Derived measurements

Compute:

- shoulder height difference
- hip height difference
- head forward offset
- torso angle
- pelvis/torso relationship
- knee-inward deviation
- center sway
- side-to-side loading imbalance

10.4 Example detections

Forward head posture

Side-view head sits too far anterior to shoulder line.

Rounded shoulders

Shoulder positioning suggests protraction / upper-body collapse pattern.

Anterior pelvic tilt tendency

Side-body pattern shows rib flare + pelvis/torso relationship consistent with APT tendency.

Knee valgus

During squat, knees drift inward relative to hip-foot line.

Asymmetry

Shoulders, hips, or movement path show left-right imbalance.

11. Recommendation logic

Each issue maps to one or more drills.

Example mappings

Rounded shoulders

Recommend:

- doorway pec stretch
- wall angel

Forward head posture

Recommend:

- wall angel
- posture reset cues

APT tendency

Recommend:

- hip flexor stretch
- pelvic reposition cues

Knee valgus

Recommend:

- squat alignment drill
- split squat alignment drill

General asymmetry

Recommend:

- single-side control drill
- split squat control
- balance drill

12. Live coaching logic

This is the most important part of the product.

Live loop

For each frame:

1. read pose landmarks
2. compute drill-specific metrics
3. compare against target thresholds
4. classify form as green/yellow/red
5. surface highest-priority cue
6. draw overlays

Cue priority

1. Unsafe warning

Example:

- “stop — knee collapsing inward”

2. Major correction

Example:

- “hips too far forward”

3. Fine adjustment

Example:

- “lift chest slightly”

This keeps the UI clean.

13. Visual system

Overlays to render

- body skeleton
- alignment lines
- joint highlights
- strain zones
- directional arrows
- cue cards
- status color

Color logic

- green = good
- yellow = needs correction
- red = unsafe / unstable

High-value visuals

Alignment lines

- shoulder line
- hip line
- spine line
- knee-over-foot line

Correction arrows

- move knee outward
- tuck pelvis
- pull head back
- shift hips back

Joint strain highlights

- neck
- shoulders
- hips
- knees

These make the demo look cracked.

14. Core architecture

Frontend

Use:

- React + Vite

Camera + CV

Use:

- webcam via `getUserMedia`
- MediaPipe Pose

Overlay rendering

Use:

- HTML canvas

Logic

Use:

- TypeScript utility modules
- rule-based detection and cue logic

Backend

Optional only.

Probably not needed for MVP unless you want saved reports.

15. Ethics and safety plan

Since this is physical health, you need clear boundaries. The hackathon specifically mentions medical accuracy, privacy, and transparency around limitations.

Product safety statement

StrainSense is:

- a corrective movement support tool
- not a diagnostic tool
- not a replacement for physical therapy or medical care

User warnings

Tell users:

- stop if pain becomes sharp or severe
- seek professional care if symptoms worsen
- use only for educational corrective guidance

Privacy

Best stance:

- all analysis happens in-browser
- video is not stored by default

That is a strong talking point.

16. The 6 broader tasks — detailed version

This is the section you actually wanted expanded.

TASK 1 — Pose Capture + Guided Scan Pipeline

Owner: Person 1

Goal

Build the entire camera and pose-input system that captures the user's body in real time and guides them through the assessment movements.

Why it matters

Nothing else exists without this. This is the raw input layer for the whole product.

What this task includes

- webcam integration
- MediaPipe Pose integration
- live landmark extraction
- full-body detection validation
- camera calibration checks
- guided assessment flow
- capture of assessment frames / sequences

Detailed subtasks

1. Webcam and permissions

- request webcam permission
- render live video feed
- handle permission rejection
- ensure video resizes correctly across devices

2. MediaPipe Pose setup

- integrate MediaPipe Pose
- get landmarks every frame
- confirm sufficient frame rate
- expose pose data to rest of app

3. Calibration logic

- detect whether full body is visible
- detect if user is too close / too far
- detect if user is off-center
- surface setup prompts:
 - "step back"
 - "move into frame"

- “improve lighting”

4. Guided assessment sequence

Create a flow for:

- front neutral stance
- side stance
- arm raise
- squat
- single-leg balance

Each step should include:

- instruction text
- countdown
- capture window
- landmark recording

5. Assessment data packaging

Output a structured object containing:

- pose landmarks per step
- timestamps
- selected representative frames
- movement sequence data

Deliverables

- working webcam feed
- real-time skeleton tracking
- calibration prompts
- guided scan flow
- structured pose data saved for analysis

Acceptance criteria

This task is done when:

- user can complete the full scan
- pose landmarks are captured for each movement
- full-body visibility is enforced
- downstream analysis can consume the data

Suggested files/modules

- `camera.ts`
 - `poseEngine.ts`
 - `assessmentFlow.ts`
 - `calibration.ts`
-

TASK 2 — Posture / Asymmetry / Instability Analysis Engine

Owner: Person 1

Goal

Convert raw pose landmarks into meaningful detected issues.

Why it matters

This is the “brain” behind the assessment phase.

What this task includes

- angle calculations
- asymmetry scoring
- instability heuristics
- issue classification
- severity / confidence labels
- evidence frame selection

Detailed subtasks

1. Landmark-derived metrics

Build functions to compute:

- shoulder level difference
- hip level difference
- head forward offset
- torso lean angle
- knee tracking deviation
- body-center shift
- sway during balance
- squat path deviation

2. Issue heuristics

Implement rule sets for:

- rounded shoulders
- forward head posture
- APT tendency
- knee valgus
- left-right asymmetry

Each issue should return:

- issue name
- confidence / severity
- explanation string
- related joints / body areas

3. Evidence frame selection

For each issue:

- identify the strongest frame or moment where deviation is visible
- tag that frame for annotation

This is important for the results page.

4. Structured analysis output

Produce a normalized object like:

- detected issues array
- per-issue metrics
- evidence frame ID
- highlighted joints
- recommended next step tag

Deliverables

- metrics engine
- issue detection logic
- severity/confidence labels
- evidence frame extraction
- clean output for results UI and recommendation engine

Acceptance criteria

Done when:

- supported issues are reliably flagged
- outputs are structured and consistent
- the rest of the app can render issue cards from the analysis

Suggested files/modules

- `metrics.ts`
 - `issueDetectors.ts`
 - `analysisEngine.ts`
 - `evidenceSelector.ts`
-

TASK 3 — Corrective Plan and Drill Recommendation Engine

Owner: Person 2

Goal

Translate detected issues into a clean, personalized corrective plan.

Why it matters

This is what makes the product useful, not just technically cool.

What this task includes

- issue-to-drill mapping
- explanation copy
- recommendation ranking
- drill metadata
- plan generation

Detailed subtasks

1. Drill library design

Build a structured drill library containing:

- drill name

- what it helps with
- target issue(s)
- main body area
- required position
- live coaching cues
- unsafe conditions
- short description
- demo asset reference

2. Mapping logic

For each detected issue:
map to 1–2 recommended drills.

Example:

- rounded shoulders → doorway pec stretch, wall angel
- APT tendency → hip flexor stretch
- knee valgus → squat tracking drill, split squat drill

3. Prioritization

If user has multiple issues:

- rank them by severity
- rank drills by importance
- avoid overwhelming with too many recommendations

4. Explanation cards

Generate text such as:

- what was detected
- why it matters
- what it may contribute to
- why the drill was chosen

Deliverables

- drill library JSON / TS config
- issue-to-drill mapping engine
- prioritized corrective plan generator
- user-facing explanations

Acceptance criteria

Done when:

- results page can show issue cards
- each issue has meaningful drill recommendations
- selected drill feels logically linked to the user's body pattern

Suggested files/modules

- `drills.ts`
 - `recommendationEngine.ts`
 - `issueCopy.ts`
 - `planBuilder.ts`
-

TASK 4 — Live Coaching + Safety Feedback Engine

Owner: Person 2

Goal

Make the system actually coach the user while they perform corrective movements.

Why it matters

This is the single most memorable feature in the whole product.

What this task includes

- drill-specific threshold logic
- live form scoring
- real-time cue generation
- unsafe state detection
- rep / hold tracking

Detailed subtasks

1. Drill-specific success metrics

For each drill, define:

- target joint positions
- acceptable angle ranges
- required body orientation

- stable hold conditions

2. Error detection

For each drill, detect common mistakes.

Example:

For hip flexor stretch:

- pelvis too far forward
- chest collapsing
- insufficient extension

For squat drill:

- knees caving inward
- torso leaning too far forward
- weight shifting to one side

3. Cue generation

For each error:

define a live correction cue.

Examples:

- “push knees outward”
- “bring ribs down”
- “move hips slightly back”
- “lift chest”
- “head drifting forward”

4. Safety logic

Define red-state conditions:

- knee collapse exceeds threshold
- excessive trunk instability
- balance failure in a held drill

Then surface:

- red highlight
- warning text
- optional “pause and reset”

5. Quality scoring

Create a lightweight score based on:

- time in acceptable range
- stability
- alignment quality
- correction success

Deliverables

- drill thresholds
- form error rules
- live cue engine
- safety alert engine
- session scoring logic

Acceptance criteria

Done when:

- user can perform at least 2 drills live
- system catches wrong form and gives feedback
- system visibly improves when user corrects posture

Suggested files/modules

- `liveCoach.ts`
- `cueEngine.ts`
- `safetyRules.ts`
- `drillThresholds.ts`
- `qualityScore.ts`

TASK 5 — Frontend UX, Flow, and Product Experience

Owner: Person 3

Goal

Build the full app flow so the product feels coherent, polished, and easy to demo.

Why it matters

Even cracked logic loses if the product experience is messy.

What this task includes

- app layout
- navigation
- onboarding
- camera setup screens
- results page
- drill selection page
- session summary page
- state management

Detailed subtasks

1. App structure

Create screens for:

- landing / intro
- camera setup
- guided assessment
- analysis loading
- results page
- drill detail page
- live coaching page
- session completion page

2. Product copy

Write clear text for:

- instructions
- issue explanations
- buttons
- warnings
- loading states
- completion states

3. State management

Track:

- assessment progress
- captured data
- detected issues

- selected drill
- live coaching state
- session quality score

4. UX polish

Ensure:

- clear transitions
- minimal clutter
- one obvious CTA per screen
- responsive layout
- demo-friendly flow

Deliverables

- working multi-screen app
- coherent navigation flow
- all main UI states
- results and drill pages wired in

Acceptance criteria

Done when:

- someone can start the app, complete assessment, see results, choose a drill, and enter live coaching without confusion

Suggested files/modules

- `App.tsx`
- `routes.ts`
- `AssessmentPage.tsx`
- `ResultsPage.tsx`
- `DrillPage.tsx`
- `CoachingPage.tsx`

TASK 6 — Visual Overlay, Annotation, and Demo Polish Layer

Owner: Person 3

Goal

Make the whole product look technically insane and instantly understandable.

Why it matters

This is what converts “cool project” into “holy shit” during the demo.

What this task includes

- skeleton rendering style
- alignment lines
- highlighted joints
- issue annotations
- arrows
- strain map visuals
- red/yellow/green visual feedback
- polished camera overlay UI

Detailed subtasks

1. Skeleton styling

Render:

- joint dots
- limb lines
- consistent colors
- active target emphasis

2. Alignment overlays

Draw:

- shoulder line
- hip line
- spine line
- knee-over-foot line

3. Joint highlights

Create dynamic highlights:

- red unstable knees
- orange tight shoulders
- yellow neck strain area

4. Correction arrows

Render arrows for:

- move knee outward
- tuck pelvis
- move head back
- shift weight left/right
- bring chest up

5. Annotated assessment visuals

On results page:

- show evidence frame
- draw highlighted issue region
- label what is wrong

6. Status polish

Create:

- green/yellow/red states
- pulsing warning glow
- clean cue box styles
- hold timer / rep display

Deliverables

- polished overlay system
- annotated result snapshots
- strong visual language
- demo-ready live coaching screen

Acceptance criteria

Done when:

- judges can understand the system visually without needing long explanation
- bad posture visibly “lights up”
- corrections are obvious and satisfying

Suggested files/modules

- `overlayRenderer.ts`
- `skeletonCanvas.ts`

- `annotations.ts`
 - `arrowRenderer.ts`
 - `statusStyles.ts`
-

17. Team split summary

Person 1

- Task 1: Pose Capture + Guided Scan Pipeline
- Task 2: Posture / Asymmetry / Instability Analysis Engine

Person 2

- Task 3: Corrective Plan + Drill Recommendation Engine
- Task 4: Live Coaching + Safety Feedback Engine

Person 3

- Task 5: Frontend UX / Flow / State
- Task 6: Visual Overlay / Annotation / Demo Polish

That is the cleanest split.

18. Dependency flow

Build order

1. Task 1 first
2. Task 2 second
3. Task 3 in parallel once issue output format is defined
4. Task 5 starts early with placeholder data
5. Task 4 begins once at least one drill is defined
6. Task 6 runs throughout as overlay polish grows

Dependency map

- Task 2 depends on Task 1
 - Task 3 depends on Task 2
 - Task 4 depends on Task 1 and Task 3
 - Task 5 depends on outputs from 2, 3, and 4
 - Task 6 depends on 1, 2, and 4
-

19. Recommended MVP implementation sequence

Milestone 1

Get webcam + skeleton working.

Milestone 2

Detect one thing convincingly:

- knee valgus in squat
or
- forward head posture

Milestone 3

Generate one issue card + one recommendation.

Milestone 4

Build one live drill coaching flow.

Milestone 5

Add overlays and arrows.

Milestone 6

Expand to 4 issue types + 4 drills.

That is the correct order.

20. Demo plan

Live demo sequence

Part 1: Assessment

Show:

- front stance
- side stance
- squat

System flags:

- rounded shoulders
- APT tendency
- knee valgus

Part 2: Results page

Show:

- issue cards
- annotated frame
- body strain visuals
- recommended drills

Part 3: Live coaching

Select:

- hip flexor stretch or squat drill

Then:

- intentionally do it wrong
- system lights up red

- arrow appears
- cue text appears

Then:

- fix form
- system turns green

That's the magic moment.

21. Judge pitch

Opening

“Most people know they feel tight, imbalanced, or in pain, but they don’t know what movement pattern is actually causing it.”

Problem

“Current solutions are generic videos, generic stretching apps, or expensive professional care with no real-time support in between.”

Solution

“StrainSense is a real-time movement intelligence system that uses computer vision to detect posture and instability issues, generate a personalized corrective plan, and coach users live with visual overlays, correction arrows, and safety alerts.”

Impact line

“We’re not just recommending stretches. We’re showing users where their body is unstable, why it matters, and how to fix it before strain becomes injury.”

Ethical line

“We intentionally scoped this as a non-diagnostic corrective movement tool with clear safety boundaries, not a replacement for physical therapy or medical care.”

22. Final product statement

StrainSense is a real-time corrective movement and posture intelligence platform that uses computer vision to analyze body mechanics, detect asymmetry and instability, generate a personalized corrective mobility plan, and coach users live with visual overlays, correction cues, and safety alerts while they perform drills and stretches.

Shorter:

We built a system that sees where your body is unstable and helps you correct it live before strain becomes injury.